

Claude Code for Beginners

Getting real work out of an AI agent, starting from zero

Version 1.0 · July 2026

By Jared Hebb · jaredhebb.com

"Claude" and "Claude Code" are trademarks of Anthropic, PBC. This is an independent course, not affiliated with or endorsed by Anthropic. © 2026 Jared Hebb. Licensed for the personal use of the purchaser. Please don't redistribute.

Claude Code for Beginners

Getting real work out of an AI agent, starting from zero

Course version: 1.0 (July 2026) **Author:** Jared Hebb **Format:** 8 self-paced modules, about 6 hours total, text lessons with hands-on exercises **Also works as:** a 1-day or 2-evening in-person workshop (see the Instructor Guide)

Who this course is for

People who keep hearing about AI coding agents and want to actually use one. Small business owners. Freelancers. Analysts. Students. You do not need to be a programmer. I'm not one, and I wrote the course. Developers new to Claude Code will move faster through the early modules but still get their money's worth from Modules 5 through 7.

What you need: a computer running Windows, Mac, or Linux, the willingness to type into a terminal, and a paid Claude plan. Pro is about \$20 a month and includes Claude Code. That's the whole shopping list.

What you'll be able to do by the end

1. Install and run Claude Code on your own machine
2. Run a productive working session: give clear instructions, review what Claude did, correct course when needed
3. Use Claude Code for everyday non-coding work: organizing files, processing data, drafting documents, automating repetitive tasks
4. Build and publish a real, live website with no coding experience
5. Use git and GitHub through Claude Code without memorizing a single command
6. Customize Claude Code with a CLAUDE.md file, slash commands, and permission settings that match your comfort level
7. Undo anything. Nothing you break stays broken.

Course map

#	Module	Time	You'll walk away with
1	What Claude Code Is (and Isn't)	30 min	A correct mental model of an AI agent
2	Setup: Install and First Launch	30 min	Claude Code running on your machine
3	Your First Real Session	45 min	The core loop: instruct, review, refine
4	Everyday Superpowers (No Code Required)	60 min	Five real tasks automated on your own files
5	Projects, Context, and CLAUDE.md	45 min	A project folder Claude understands
6	Git and GitHub Without Tears	45 min	Version-controlled work you can always undo
7	Making It Yours: Permissions, Commands, Customization	45 min	A setup tuned to how you work
8	Capstone: Build and Publish a Website	90 min	A live website on the internet
9	Bonus: A Business Back Office on Claude Agents	30 min	A real case study, agent files included, plus a pattern to build your own

How to take this course

Do the exercises. All of them. Here's the truth about courses: reading feels like progress and produces nothing. Every module ends with hands-on work, and the exercises are where the actual learning lives. Reading teaches you *about* Claude Code. The exercises teach you Claude Code. Those are different products, and only one of them was worth paying for.

Use your own real files and problems wherever an exercise allows it. My examples make the course make sense. Your problems make it pay.

Keep the cheat sheet open (`cheatsheet.md`). One page, everything in the course.

Stuck? Ask Claude Code itself. "Explain what just happened" is a completely legitimate move, and Module 3 teaches you to use it.

What this course is not

It's not a programming course. You'll ship working code without reading much of it. Module 1 explains why that's fine for beginners and exactly where the limits are.

It's not a bag of prompt tricks. Tricks expire. The skill you're building is delegation: scoping work, reviewing results, iterating. Delegation doesn't expire.

It's not tied to one version of Claude Code. The tool updates constantly, so the course teaches what stays stable and points you to `/help` and the official docs for what changes fast.

Module 1: What Claude Code Is (and Isn't)

Time: about 30 minutes. **Hands-on:** thought exercises only. Setup comes in Module 2.

Objectives

By the end of this module you can:

- Explain the difference between a chatbot and an agent
 - Tell Claude, Claude Code, and Claude Cowork apart, and say why this course teaches Claude Code
 - Describe what Claude Code can touch on your computer and what it can't
 - Name the three habits that separate productive users from frustrated ones
-

1.1 Chatbot vs. agent

You've used a chatbot. You type a question, it types an answer. The conversation lives in a browser tab, and if you want to actually *do* anything with the answer (save a file, rename 200 photos, update a spreadsheet) you do it yourself. By hand. The chatbot talks about the work. You still do the work.

Claude Code is a different animal. It's an agent. It runs on your computer, in your terminal, and it takes actions:

- read and write files on your machine
- run programs and commands
- search the web
- use git, install software, call APIs

You describe an outcome, something like "organize my Downloads folder by file type and year," and it plans the steps, executes them, checks the results, and reports back. When something fails, it reads the error message and tries another approach, the way a competent assistant would.

Here's the mental model that works, and I want you to actually adopt it, not just nod at it: **Claude Code is a very fast, very literal contractor you just hired.** Skilled. Never tired. But it only knows what's in front of it: your instructions, your files, whatever it discovers along the way. Good outcomes come from a good brief: clear goal, relevant context, known constraints. That's true of every contractor you've ever hired, and it's true here.

The mental model that fails: mind reader. "Fix my website" gets you clarifying questions or a guess. "The contact form on my site returns an error after clicking submit. The project folder is right here. Find the cause and fix it" gets you a fix. Same tool. Same day. The variable was you.

1.2 Claude, Claude Code, and Cowork: which one is this?

Anthropic sells one subscription with three doors into the same brain, and the names confuse everyone. Thirty seconds to sort it out.

Claude (the app and website) is the chatbot from section 1.1. It talks. You do.

Claude Cowork lives in the Claude desktop app, and it's rolling out on the claude.ai website and phone apps too. It's an agent like Claude Code, but with buttons instead of a terminal: point it at a folder, and it can organize files, pull data out of PDFs, build reports, and run scheduled tasks. It launched in early 2026 and it's included in paid plans, same as Claude Code. If all you ever want is document and file chores, Cowork alone might cover you, and I'd rather tell you that now than have you find out after buying a course.

Claude Code is the full toolbox. Everything Cowork does, plus the things this course is actually about: building and publishing a real website, the git undo button that makes mistakes free (Module 6), automation you set up once and rerun forever, and a workspace you customize per project (Module 5 and 7). Cowork drives an automatic; Claude Code hands you the keys to the truck.

So why does this course teach the terminal version? Because the terminal is a one-week discomfort and the toolbox is permanent. Modules 2 and 3 get you through the discomfort. The rest of the course spends the payoff. And once you're comfortable in Claude Code, Cowork will feel obvious on the days you want the simpler door.

1.3 What it can and can't touch

Claude Code operates from a working directory, the folder you launch it in. Think of it as the job site.

It can read, edit, and create files, run terminal commands, use the internet, work with git repositories, and (with your permission) act outside the working directory.

It can't act without being run. It can't see your screen. It can't click around inside apps like Excel or Photoshop, because it works with files, not windows. And it can't do anything your user account can't do.

Permissions are your safety rail. Out of the box, Claude Code asks before doing anything with side effects: editing a file, running a command. You'll see a prompt: allow once, allow always, or deny. Early on, read these prompts. Every one. They're how you learn what the tool actually does, and that education is free. Later, in Module 7, you'll tune them so routine actions stop nagging you.

1.4 The three habits

Put a frustrated beginner and a productive user side by side and you'll see the same three differences every single time. Not ten. Three. Master these and you're ahead of almost everyone who installs this tool.

Habit 1: Scope the work. One coherent outcome at a time. "Clean up this folder, then build me a website, and also can you look at my taxes" produces mush. One brief, one outcome, next.

Habit 2: Review before you build on top. After Claude finishes, look at what changed. You don't need to read code line by line. Ask: "What did you change and why?" Then check the result itself: open the file, load the page, look at the report. Skipping this step is how a small problem two weeks ago becomes a big problem today.

Habit 3: Correct course instead of starting over. Result is 80% right? Say what's wrong with the other 20%: "Good, but the dates should be DD/MM/YYYY, and skip hidden files." Iterating in the same conversation keeps all the context Claude has built up. Restarting throws it away.

1.5 "But I can't read code. Isn't this dangerous?"

Fair question. Three straight answers.

First, most beginner tasks aren't code. Organizing files, summarizing documents, cleaning up CSVs. You verify the outcome with your own eyes, no code reading required.

Second, version control is your undo button. Module 6 sets it up, and after that, any change Claude makes to a project rolls back with one sentence: "undo the last change." When mistakes cost nothing, fear stops being useful.

Third, know where the boundary is. Anywhere a silent mistake is expensive (payroll data, legal documents, systems other people depend on), you verify the output independently or bring in someone who can. The course flags these moments explicitly when they come up. That's not an AI rule. That's a business rule.

1.6 What it costs

Claude Code is included in paid Claude plans. Pro, around \$20 a month, covers this entire course comfortably. Heavy professional use can eventually hit plan limits, and the fix is a bigger plan or API billing, but that's a decision you won't face until long after this course. Twenty bucks a month against the hours you're about to reclaim. Run that math once and move on.

Exercises

1. Write your first three briefs. Pick three real annoyances from your work or computer life: a messy folder, a repetitive weekly task, a document you keep meaning to make. For each one, write a two-to-four sentence brief as if handing it to a contractor: the outcome, where the materials are, the constraints. Keep these. You'll run one in Module 3. This is the ten-minute exercise that triples the value of everything after it. Do it.

2. Sort these requests. For each one, decide: good brief, needs scoping, or needs extra care in verification?

- "Make my resume better"
- "Rewrite my resume (resume.docx on my Desktop) to emphasize project management, and keep it to one page"
- "Update the pricing formulas in our client invoicing spreadsheet"

Answers: the first needs scoping (better how? for what job?). The second is a good brief. The third needs verification care: money math gets an independent check, always. One way: ask Claude to also produce a before-and-after comparison you can eyeball.

Checkpoint

You're ready for Module 2 if you can answer these cold:

- What's the difference between an agent and a chatbot? (An agent takes actions on your machine. A chatbot produces text in a tab.)
- When would Cowork alone cover someone? (When all they need is file and document chores. Claude Code adds building, publishing, the git undo button, and per-project setup.)
- What's a working directory? (The folder Claude Code is launched in. Its job site.)
- What are the three habits? (Scope the work, review the results, correct course instead of restarting.)

Module 2: Setup, Install, and First Launch

Time: about 30 minutes. **Hands-on:** yes. You'll finish with Claude Code running.

Objectives

- Open a terminal on your operating system without dread
 - Install Claude Code and log in
 - Launch it in a safe practice folder and run your first commands
-

2.1 Meet your terminal

The terminal is the thing that stops most people from ever starting. So let's kill the fear with a fact: you need exactly three ideas from it. Three. Not thirty.

1. You type commands, press Enter, they run.
2. You're always "in" a folder (the current directory). `cd foldername` moves you into a folder. `cd ..` moves you up one level.
3. Copy and paste work. Windows: Ctrl+Shift+V or right-click. Mac: Cmd+V.

To open it:

- **Windows:** press Start, type `PowerShell`, press Enter.
- **Mac:** press Cmd+Space, type `Terminal`, press Enter.
- **Linux:** you already know.

Try it right now. Type `cd Desktop` and press Enter. Done? Congratulations, that's most of the terminal skill this entire course requires. The scary black window took you eleven seconds. Claude Code handles the rest.

2.2 Install Claude Code

You need a paid Claude account first (Pro or higher). Sign up at claude.ai if you haven't.

For the install itself, check <https://docs.anthropic.com/claude-code> for the current command for your operating system. Install methods change over time and the docs are always right. As of this writing:

- **Windows (PowerShell):** `irm https://claude.ai/install.ps1 | iex`
- **Mac/Linux:** `curl -fsSL https://claude.ai/install.sh | bash`

Close and reopen your terminal after installing, then verify:

```
claude --version
```

Version number? You're in. "Command not found"? Reopen the terminal and try again, that fixes it nine times out of ten. Still stuck? The docs page has troubleshooting per OS. The usual culprit is something called PATH, and here's the fun part: once you're running, diagnosing PATH problems is exactly the kind of thing you'll hand to Claude Code.

2.3 First launch

Rule one of new tools: don't test them where you keep things you love. Make a sandbox:

```
mkdir claude-practice
cd claude-practice
claude
```

The first run walks you through login (it opens a browser to authenticate with your Claude account) and a couple of preference questions. Defaults are fine.

You land at a prompt. This is a conversation. Type in plain English:

```
Create a file called hello.txt containing a haiku about terminals.
```

Two things happen. First, Claude Code asks permission to write the file, the permission prompt from Module 1, live and in person. Approve it. Second, the file appears. Now verify it: type "Read hello.txt back to me," or open the folder in your file explorer and look with your own eyes.

Stop and notice what you just did, because it's the whole course in miniature: **instruct, permission, action, verify**. Everything from here is that loop with bigger stakes.

2.4 Five things worth knowing on day one

Inside a Claude Code session, lines starting with `/` are commands to the tool itself rather than messages to Claude:

Command	What it does
<code>/help</code>	Lists everything Claude Code can do. The always-current manual.
<code>/clear</code>	Wipes the conversation and starts fresh. New task, new clear.
<code>/model</code>	Shows or changes which Claude model you're using
<code>/config</code>	Settings: theme, notifications, and so on
<code>Esc</code> (the key)	Interrupts Claude mid-action. Your brake pedal.

To leave, type `/exit`, or press `Ctrl+C` twice (the first one just cancels whatever you're typing; you'll see "Press Ctrl-C again to exit"). Your files stay. Only the conversation ends. Relaunching with `claude --continue` picks up your most recent conversation in that folder.

2.5 Where to work from

You don't have to `cd` into the right folder every time. Claude Code will launch from wherever your terminal already is, and reach outside that folder when a task calls for it. The permission prompt is the real safety net here, not the folder you happened to launch from: anything outside the working directory, or anything destructive, stops and asks first.

That said, `cd`-ing into the project folder before you start is still a good habit when it's easy. It keeps you oriented, and "everything below here" becomes obvious shared context for both of you. Cleaning up Downloads? `cd Downloads` first. Working on a website project? `cd my-website`. In a hurry, or bouncing between a few things? Just launch and let the permission prompt do its job.

Exercises

1. **Install and verify.** `claude --version` shows a number.
2. **The full loop, three times.** In your sandbox, ask for: (a) a file listing 5 dinner ideas, (b) a second file translating them into another language, (c) both files combined into one nicely formatted `menu.md`. Approve the permissions and verify each result yourself in your file explorer. Reps build the habit. The habit builds the trust.

3. **Use the brake.** Ask Claude to "count from 1 to 1,000,000 in a file" and press `Esc` once it starts. Nothing breaks. Interrupting is allowed, learn that now, while the stakes are a joke file. Then ask it to delete that file.
4. **Ask the tool about itself.** Run `/help` , then ask in plain English: "What can you do with images?" Interrogating the tool is a skill that pays forever, because the tool keeps changing and the skill doesn't.

Common snags

- "command not found" right after installing: close and reopen the terminal.
- Login loop, or the browser doesn't open: copy the printed URL into your browser manually.
- Nervous about permission prompts: for this whole course, when in doubt, read what it's asking. If the action is inside your sandbox or project folder, approve it. Module 7 covers loosening and tightening this deliberately.

Checkpoint

You're ready for Module 3 when Claude Code launches, you've created and verified real files with it, and you know how to interrupt (`Esc`) and exit (`/exit` , or `Ctrl+C` twice).

Module 3: Your First Real Session

Time: about 45 minutes. **Hands-on:** yes. You'll run one of your own briefs from Module 1.

Objectives

- Run the instruct, review, refine loop on a real task
 - Write briefs that get good results on the first try
 - Recover confidently when a result is wrong
-

3.1 Anatomy of a good brief

A brief that works has four parts. You won't always need all four, but when a result disappoints you, come back to this list, because one of these is what was missing. Every time.

1. **Outcome.** What exists when the work is done. "A single PDF combining these three reports."
2. **Materials.** Where the inputs are. "The reports are the three .docx files in this folder."
3. **Constraints.** What must or must not happen. "Keep the original files untouched. Order: Q1, Q2, Q3."
4. **Definition of done.** How you'll both know it worked. "Tell me the final page count so I can sanity-check it."

Compare:

"combine my reports"

"Combine the three quarterly .docx reports in this folder into one PDF named 2026-annual.pdf , in Q1 to Q3 order, leaving the originals untouched. Tell me the final page count."

The second version costs you 20 extra seconds of typing. It saves you three rounds of back and forth. That trade, seconds of clarity for minutes of correction, is the highest-ROI trade in this entire course, and you get to make it on every single task forever.

One more thing. Notice the brief says nothing about HOW to convert docx files to PDF. Not your job. If a tool needs installing, Claude proposes it. **You own the what. Claude owns the how.** Write that somewhere.

3.2 What you're looking at while Claude works

During a task, Claude narrates: reading files, running commands, making edits. You don't need to follow every line. Watch for three things:

- **Permission prompts.** Read them. They tell you what's about to happen before it happens.
- **Questions.** Claude asks when your brief left a real decision open. Answering well beats letting it guess.
- **The wrap-up summary.** Claude's account of what it did. Your review starts here.

Heading somewhere you don't like? Press `Esc` and redirect. Interrupting is normal usage, not an emergency measure. You're the boss. Act like it.

3.3 Reviewing without expertise

Three review moves, cheapest first:

1. **Check the artifact.** Open the file. Load the page. Look at the folder. Most beginner tasks verify with your eyes in under a minute.
2. **Interrogate.** "Summarize every file you changed and why." "What could go wrong with this approach?" "Did anything fail that you worked around?" Claude answers honestly about its own work, including its doubts. Free information. Take it.
3. **Ask for a self-check.** "Write a quick check that verifies the totals in the output match the input files, and run it." A second method catching the first is how silent errors die.

Money, legal language, anything sent to other people: do move 3, then add your own independent look on top. Every single time. No exceptions. No matter how good the last ten results were.

3.4 Refining: the 80% pattern

First results are commonly 80% right. Beginners see the 20% and start over. That's the expensive move. The winning move is a targeted correction:

"Good. Two fixes. First, the dates should read like 'July 2, 2026', not '02/07/26'. Second, you missed the files in the subfolder, so include those too."

Why not restart? Because the session has context: everything Claude has read and learned so far. A restart sets that on fire. Two rules:

- Same task? Keep going in one conversation.
- New unrelated task? `/clear` first. Stale context from an old task confuses a new one, and very long conversations lose quality. New job, fresh slate.

3.5 When it's really wrong

Sometimes iteration isn't converging. Don't flail. Climb the ladder:

1. Say what's wrong specifically, with an example of the wrong output next to what you expected.
2. Separate diagnosis from action: "Stop fixing. First explain why the totals are off, then propose a fix and wait for my okay." This is the single most useful sentence for stuck situations. Steal it.
3. Undo and re-brief: "Restore everything to how it was before this task." Then `/clear`, and write a better brief using what you learned. Before Module 6, Claude can usually reverse its own file changes. After Module 6, undo becomes trivial and guaranteed.

A failed task that teaches you what the brief was missing isn't a failure. It's tuition, and it's a lot cheaper than mine was.

3.6 A worked example

A brief given in a folder of 340 vacation photos:

"Rename all photos in this folder to `YYYY-MM-DD_HHMM_original-name.jpg` using each photo's taken-date from its metadata. Photos with no metadata date go untouched into a subfolder called `no-date`. Show me a plan and a 5-file sample first."

Why it's strong: outcome, materials (this folder), constraints (how to handle photos without dates). And a staged rollout: "plan and sample first." For any task touching a lot of files, a small sample before the full run turns a potential 340-file mistake into a 5-file one. Same insurance a smart contractor sells you, except this one's free.

Exercises

1. **Run one of your Module 1 briefs.** Pick the safest of the three. Before running it, upgrade it with the four parts (outcome, materials, constraints, done), and if it touches many files, add a sample-first stage. Work in a copy of the real folder this first time.
2. **Practice the interrogation.** After it finishes, ask all three review questions from 3.3 and actually read the answers.
3. **Force a refinement.** Find something imperfect in the result (there's usually something) and fix it with a targeted correction, not a restart.
4. **Practice the undo.** Ask: "Restore this folder to exactly how it was before we started." Then verify it did. Undo skills learned in a crisis are learned badly. Learn them now, while nothing's on fire.

Checkpoint

You're ready for Module 4 when you've completed a real task of your own end to end, refined it at least once without restarting, and undone at least one change on purpose.

Module 4: Everyday Superpowers (No Code Required)

Time: about 60 minutes. **Hands-on:** five guided mini-projects.

Objectives

- Recognize the five kinds of tasks where Claude Code pays for itself immediately
 - Complete one hands-on task in each category
 - Start noticing, during your normal week, "wait, this is a Claude Code task"
-

This is the module that pays for the course, so let's be clear about the unlock. It isn't a skill. It's recognition. The tool can already do the work. The bottleneck is that most people never notice the tedious thing they're doing by hand could be delegated. They just... keep doing it. For years. This module installs the noticing reflex, using five task families.

4.1 Family 1: File wrangling

Anything involving lots of files: renaming, sorting, deduplicating, converting formats, restructuring folders.

Mini-project A: tame a messy folder. Launch Claude Code in your Downloads folder (or a copy of it) and run:

"Organize this folder: create subfolders by file type (Documents, Images, Installers, Archives, Other) and move files accordingly. Within Documents, make year subfolders based on each file's modified date. Show me the plan and move 10 files as a sample before doing the rest. Don't delete anything."

Notice the safety habits baked into the brief: an explicit "don't delete anything," and a sample before the full run. That's not paranoia. That's how you get to be bold cheaply.

4.2 Family 2: Data and spreadsheets

CSVs, Excel exports, merging sheets, cleaning inconsistent data, computing summaries, making charts. Claude Code handles these with real tools. Under the hood it quietly uses Python or similar, and you never have to care.

Mini-project B: clean and summarize. Get any CSV. A bank export or a sales report works, or ask Claude to generate a realistic messy sample CSV of 500 fake orders. Then:

"Clean this CSV: standardize the date format, trim whitespace, and flag duplicate rows in a new column rather than deleting them. Then give me a summary: totals by month, top 5 categories, and anything weird you noticed. Save the cleaned file separately. Never overwrite the original."

The phrase "anything weird you noticed" earns its keep every time. Agents catch anomalies humans skim past. It's five words. Use them.

4.3 Family 3: Documents and writing

Drafting from bullet points, reformatting, merging documents, generating variations (say, 12 personalized versions of one letter), extracting information from PDFs.

Mini-project C: mail merge without Mail Merge. Create a small CSV of 5 names and companies (or have Claude invent them), plus a paragraph describing something you're offering. Then:

"Using contacts.csv and my offer description in offer.txt, draft a personalized outreach email for each contact as separate .txt files. The audience is small-business owners, so keep it warm and direct, no jargon, under 150 words each."

Review one closely, give tone feedback ("less salesy, more neighborly"), and have it regenerate all five. Burn that loop into your brain: **edit one, apply to all**. It's the difference between fixing a letter and fixing a system.

4.4 Family 4: Research and synthesis

Claude Code can search the web, read pages, and pull what it finds into a document. Useful when the output should be a file you keep, not a chat answer that scrolls away.

Mini-project D: a comparison table. Pick a real decision you're facing (accounting software, a standing desk, a CRM):

"Research the top 4 options for [your decision] for [your situation]. Build me a comparison table in a markdown file: price, key features, the main complaint from real users, and who each one is best for. Include links to sources so I can check the claims."

Straight talk on verification: research is the family where confident-but-wrong answers are most likely. The "include links" line isn't decoration. Spot-check any claim that would actually change your decision. Trust the table for orientation. Verify it for commitment.

4.5 Family 5: Repetitive workflows become tools

The graduation move. When you notice you'll want a task again, don't plan to re-ask. Ask Claude to turn it into a reusable script you run without thinking.

Mini-project E: your first tool. Take the Downloads-organizing task from 4.1:

"Turn the folder organizing we just did into a script I can re-run anytime. I'm on [Windows/Mac]. Make it something I can double-click, and have it print a summary of what it moved."

Read that outcome again: you now own software you didn't write. It cost you a sentence. When it needs a tweak next month, launch Claude Code and ask for the tweak. This is the pattern that turns a \$49 course into hours back every week.

4.6 The recognition checklist

You probably have a Claude Code task on your hands when the work is:

- Repetitive (the same action across many items)
- Format-shifting (this file type into that file type, this layout into that layout)
- Rule-based tedium ("if X, put it in Y," a hundred times over)
- Synthesis (many sources condensed into one document or table)
- Something you'd do if it took 5 minutes, but keep skipping because it would take an hour by hand

That last one is the money category. Everyone has those tasks. Add yours up: three hours a week is 150 hours a year. Put this list somewhere you'll see it for two weeks. The reflex forms fast, and once it forms, it compounds.

Exercises

Complete mini-projects A through E above, in any order. Then do the one that matters:

Capstone exercise: identify one recurring task from your actual work week, run it through Claude Code, and turn it into a reusable script using the Family 5 pattern. This one task, repeated weekly, pays back the course and the subscription within a month. Everything after that is margin. Don't skip this one, it's the whole reason you're here.

Checkpoint

You're ready for Module 5 when you've completed all five mini-projects and your own recurring-task capstone, and at least once this week you've caught yourself thinking "this is a Claude Code task."

Module 5: Projects, Context, and CLAUDE.md

Time: about 45 minutes. **Hands-on:** yes. You'll set up a real project folder.

Objectives

- Understand context: what Claude knows, when it knows it, and why it forgets
 - Create a CLAUDE.md so every session starts already briefed
 - Structure ongoing work as projects instead of one-off tasks
-

5.1 What Claude actually "knows"

In any session, Claude Code knows exactly three kinds of things. Three. Once you internalize this, half the confusing behavior stops being confusing:

1. **Training knowledge.** General knowledge of the world, languages, and tools. Fixed, with a cutoff date.
2. **What it reads this session.** Files it opens, commands it runs, web pages it fetches. This is called the context.
3. **What you tell it.** Your messages.

Two consequences surprise beginners, so hear them now instead of learning them the annoying way:

It doesn't automatically know your files. It discovers them by looking, which it does readily, but a brief that says where things are saves the scavenger hunt.

It forgets between sessions. Close the conversation and the context is gone. Tomorrow starts blank. (`claude --continue` resumes a past conversation, but the real fix for recurring context is the next section, and the next section is worth the price of this module.)

5.2 CLAUDE.md, your standing brief

Any folder can contain a file named `CLAUDE.md`. When you launch Claude Code in that folder, it reads that file automatically. Every session. Think about what that means: it's the briefing document a good manager hands a new contractor on day one, except you write it once and it briefs every session forever.

Here's what one looks like:

```
# Hebb Consulting: Client Reports Project
```

```
## What this project is
```

```
Monthly performance reports for 12 clients. Raw exports land in /inbox,  
finished PDFs go to /reports/<client>/<year>.
```

```
## Rules
```

- Never modify anything in /inbox. Treat it as read-only.
- All currency formatted as USD with commas: \$12,345.67
- Client names must match the spelling in clients.csv exactly.
- Drafts are always reviewed by me before anything gets emailed or sent anywhere.

```
## How I like to work
```

- Show me a plan before touching more than 5 files.
- When in doubt about a judgment call, ask. Don't guess.

Notice what's in there: facts Claude can't discover on its own (where finished work goes), standing constraints (the inbox is read-only), your working style (plan first). Notice what's not: anything Claude can figure out by looking at the files. Don't document what's discoverable. Document what's decided.

And don't write it by hand. Delegate the delegation:

"Look around this project and interview me briefly, then write a CLAUDE.md capturing what this project is, the rules I tell you, and how I like to work."

(For code projects, the built-in `/init` command does a version of this automatically.)

Maintain it conversationally. When you correct Claude on something that will always be true ("we never touch /inbox"), follow up with: "add that to CLAUDE.md." There's also a shortcut: start a message with `#` and Claude Code offers to store that note in memory for future sessions. Ten seconds now, and you never make that correction again. That's the trade. Take it every time.

5.3 Layered CLAUDE.md files

CLAUDE.md files stack:

- `~/.claude/CLAUDE.md` (in your home folder) is personal and applies everywhere: "I'm on Windows 11. I prefer plans before big changes. Explain things without assuming programming knowledge."
- A **project folder's** `CLAUDE.md` holds that project's facts and rules.
- **Subfolder** `CLAUDE.md` files exist too. You don't need them as a beginner. Ignore them guilt-free.

Set up the personal one today. Three sentences about you and your preferences, and every future session everywhere starts smarter. Three sentences. Highest leverage-per-word you'll write all week.

5.4 Structuring ongoing work

One-off tasks can live anywhere. Anything you'll touch weekly deserves a project folder:

```
my-project/  
  CLAUDE.md      <- standing brief  
  inbox/         <- raw inputs land here  
  work/          <- intermediate files  
  output/        <- finished products
```

The exact shape matters less than having a shape and recording it in `CLAUDE.md`. Here's the payoff: from then on, a session starts with "Two new client exports in inbox. Run the usual monthly report." One sentence. It works because `CLAUDE.md` is carrying the other 90% of the brief. You did the briefing work once. Now it's an asset, not a chore.

5.5 Managing context within a session

Long sessions degrade. Claude Code manages its own context as conversations grow, but running `/clear` at the start of each new task keeps quality high. Your `CLAUDE.md` files get re-read automatically after a clear. The standing brief survives, only the chat history is wiped.

`/compact` condenses a very long conversation you're not ready to leave, keeping the important parts.

Point, don't paste. Working with a big document? Don't paste it into the chat. Say the filename. Claude reads what it needs from disk.

Exercises

1. **Personal CLAUDE.md.** Create `~/ .claude/CLAUDE.md` . Ask Claude Code to do it: "create my personal CLAUDE.md, and interview me for it." Include your OS, your experience level, and two working-style preferences.
2. **Project setup.** Take your most likely recurring use (probably your Module 4 capstone). Create a project folder with the inbox/work/output shape and an interview-generated CLAUDE.md.
3. **Prove it works.** Exit, relaunch in the project folder, and give a deliberately lazy instruction ("process what's in inbox"). Watch how much the CLAUDE.md fills in. That gap between what you typed and what happened? That's your briefing asset paying dividends.
4. **Teach it one rule.** Mid-session, correct Claude about anything, then say "add that to CLAUDE.md." Confirm the file changed.

Checkpoint

You're ready for Module 6 when a fresh session in your project folder already knows your rules without being told, and you can say what belongs in CLAUDE.md (standing facts, rules, preferences) versus what belongs in the brief (the specifics of today's task).

Module 6: Git and GitHub Without Tears

Time: about 45 minutes. **Hands-on:** yes. Your project goes under version control.

Objectives

- Understand what git does for you, in one paragraph, no diagrams
 - Put a project under version control using only plain English
 - Undo changes with confidence (this is the real payoff)
 - Back up a project to GitHub
-

6.1 Git in one paragraph

Git is a save-point system for a folder. At any moment you can say "save everything as it is now" (that's called a commit) along with a note about what changed. Later you can see the history of every save, compare any two, or restore the folder, or a single file, to any past save-point. That's the whole idea. Git is famous for being complicated, but the complicated part is its command-line interface, and here's the good news: you will never learn it. Claude Code speaks fluent git. You speak English to Claude Code. Done.

Why am I making you do this before the fun capstone? Because Module 1 promised an undo button for everything, and this is it. Once a project is under git, no mistake, Claude's or yours, is permanent. And that changes the whole economics of delegation. "Try restructuring the whole site" is a terrifying request when a bad result is forever. It's a cheap experiment when the whole site is one sentence away from restored. Bold and safe at the same time. That's what you're buying with this module.

6.2 Putting a project under git

In your project folder from Module 5, say:

"Put this project under version control. Some files shouldn't be tracked. Figure out which ones (temp files, huge files, anything sensitive like API keys) and set that up too. Then make the first commit."

Claude will run `git init`, create a `.gitignore` (the do-not-track list, notice it thought about your secrets without being asked), and commit. Verify with "show me the history." You should see one commit. That's it. The thing developers take whole workshops to learn, you just did with a paragraph.

6.3 The working rhythm

The new habit is one sentence long: **commit at every good state.**

- Finished a chunk of work that's correct? "Commit this with a sensible message."
- About to attempt something big or risky? "Commit first, then let's try it."

Claude writes the commit messages. You just decide when. The rule of thumb: commit whenever you'd be annoyed to lose what you just got working. In practice that's a few times per session. Claude Code won't commit on its own initiative. You say when, and that's by design. You're the one with skin in the game.

6.4 Undo, in all its flavors

All plain English. The only vocabulary that matters is being clear about what you want undone.

You say	What happens
"Undo the changes since the last commit"	Uncommitted work is reverted and the last save-point is restored
"Undo just the changes to pricing.md, keep the rest"	Single-file restore
"Roll the whole project back to how it was Tuesday"	Time travel. Claude finds Tuesday's commit and asks whether to revert or reset, in plain terms. When unsure, say "keep history."
"What changed between now and the last commit?"	You get the differences, explained in English. Great pre-commit review.
"I think we broke something in the last few commits. Find where."	Claude walks the history comparing versions to locate the break

Practice this now, on purpose (see the exercises). Undo skills learned during a crisis are learned badly. Undo skills learned on a Tuesday afternoon with nothing at stake are learned for life.

6.5 GitHub: your project, off your machine

Git lives on your machine. GitHub hosts a synced copy online. Why you want it: backup (laptop dies, project doesn't), access from a second machine, sharing and collaboration, and later, deployment. The website in Module 8 publishes straight from GitHub.

One-time setup:

1. Create a free account at github.com.
2. Install the GitHub CLI (`gh`). Ask Claude Code to install and set it up. When the login step needs your browser, it hands you a link.
3. Then: "Create a private GitHub repository for this project and push it."

Default to private repositories. Public is for things you've deliberately decided to publish, not things you forgot to hide.

From then on, syncing is one sentence: "push my latest commits." Claude will also mention pushing when it's overdue.

6.6 What you can skip for now

Branches, pull requests, merge conflicts, rebasing. Real git topics. You need exactly none of them until you're collaborating with other people on the same files. When that day comes, ask Claude Code to teach you branches using your actual project, it's a much better tutor with a real example in front of it. Until then: one branch, commit often, push regularly. For a solo person, that's not a beginner's compromise. That's a complete, professional-grade workflow.

Exercises

1. **Version your project** (section 6.2) and verify with "show me the history."
2. **The fire drill.** (a) Ask Claude to make some visible change to a file. (b) "Undo the changes since the last commit." (c) Confirm the file is restored. (d) Make two commits with distinct small changes, then roll back to the first, then return to the latest. Pass this and you're crisis-proof. Not crisis-resistant. Proof.
3. **The diff habit.** Before your next commit, ask "what changed since the last commit?" and read the explanation. Make this your default pre-commit move.
4. **Go remote.** GitHub account, `gh` set up, private repo, push. Then open github.com and see your files sitting there.

5. **Update CLAUDE.md.** Add: "Remind me to commit when we reach a good state, and to push at the end of a session."

Checkpoint

You're ready for Module 7 when your project is on GitHub, you've completed the fire drill without help, and "commit first, then let's try it" has entered your vocabulary for risky changes.

Module 7: Making It Yours. Permissions, Commands, and Customization

Time: about 45 minutes. **Hands-on:** yes. You'll tune your setup and build a custom command.

Objectives

- Tune permissions deliberately instead of clicking through prompts
 - Use plan mode for look-before-you-leap work
 - Create your first custom slash command
 - Know what MCP, skills, and hooks are, and when you'll be ready for them
-

7.1 Permissions, deliberately

By now you've clicked through hundreds of permission prompts. Time to stop paying that tax on autopilot and start deciding. At each prompt your realistic options are:

- **Allow once**, for anything unusual.
- **Allow always**, the "stop asking about this" button, for actions you've decided you trust (like editing files inside this project). Remembered per project.

Your risk model as a beginner is one sentence: **loosen permissions in proportion to your ability to undo**. Inside a git-tracked project, file edits are cheap to undo (that was Module 6), so be generous there, it costs you almost nothing and removes most of the friction. Commands that reach outside the project or out to the internet? Those get a glance each time. Un-versioned folder of original files? Keep every prompt.

There are also two whole-session modes worth knowing. You can see the current mode in the interface, and Shift+Tab cycles through them.

Plan mode. Claude researches and proposes, and nothing gets changed until you approve the plan. Ideal for "I want to restructure this whole project," or any brief that's big or fuzzy. Get the reflex: big fuzzy brief → plan mode. It costs you one approval click and saves you entire wrong directions.

Auto-accept mode. No permission prompts for the session. Fine for low-stakes sandbox work. Think twice in folders that matter. (You'll also see `--dangerously-skip-permissions` in internet tutorials. The

name is not being cute. It's telling you exactly what it does. Not while you're new, and never outside an isolated environment.)

7.2 Custom slash commands: your recurring briefs, canned

Here's a rule that will serve you forever: any brief you've typed three times is a product waiting to be built. A file at `.claude/commands/report.md` inside your project, containing a prompt template, becomes `/report` in your sessions. Weekly task, one command, zero re-explaining.

Don't write it by hand. Delegate the meta-task:

"Create a custom slash command called `/monthly-report` that processes any new files in inbox/, follows the report rules in `CLAUDE.md`, and finishes by telling me what it produced and what looked unusual. Commands can take arguments, so let me pass a client name to run just that client."

Commands in `~/claude/commands/` work in every project, your personal toolbox. Commands in a project's `.claude/commands/` folder travel with that project, and since Module 6 they're version-controlled and pushed to GitHub like everything else. Look at what just happened: you maintain software now. Nobody told the software industry it would be this easy to get in.

7.3 The extension landscape: a map, not the territory

You'll hear these terms. Here's what each one is and, more useful, the trigger that tells you it's your moment:

Thing	What it is	Your trigger moment
Skills	Folders of instructions Claude loads for specific kinds of tasks (like "how we write proposals here"). Like <code>CLAUDE.md</code> , but per-capability instead of per-project.	A type of task, not a project, has rules worth teaching once
MCP servers	Plug-ins connecting Claude Code to outside services: Gmail, Notion, databases, your calendar	You catch yourself exporting data from a service just to hand it to Claude

Thing	What it is	Your trigger moment
Hooks	Automations that fire on events ("after every file edit, run X")	You have a rule you want enforced by the machine instead of remembered by the model
Subagents	Claude delegating pieces of work to parallel copies of itself	Mostly automatic. Ignore it.

When a trigger fires, the play is always the same: ask Claude Code to set it up. "Connect to my Notion." "Add a hook that backs up the output folder after changes." The tool configures itself under your direction. You don't read configuration documentation. You recognize the moment and ask. That's the whole game.

7.4 A note on models

`/model` picks the brain. Flagship models (Opus-class) think deepest. Lighter ones (Sonnet and Haiku class) are faster and cheaper per task. The beginner policy that works: use the default, escalate to the biggest model when a task is high-stakes or has failed twice. The model lineup changes constantly. The policy doesn't. Learn policies, not lineups.

Exercises

- Permissions audit.** In your project, deliberately set "always allow" for in-project file edits. Then ask Claude: "show me what permissions are currently saved for this project" and read the list.
- Plan mode rep.** Switch to plan mode (Shift+Tab) and give it a big fuzzy brief: "Propose a reorganization of this project for the next 6 months of use." Review the plan. Approve or revise, your call, and either way you've learned the mode. Notice that nothing was touched until you decided. That's the point.
- Build your command.** Turn your Module 4 capstone (that recurring weekly task) into a custom slash command. Test it. Commit it. Your hour-long weekly task is now a slash command. Do the annual math on that.
- One extension conversation.** Install nothing. Just ask: "Given how I work, which MCP servers or skills would actually help me? Interview me if needed." Keep the answer for the day a trigger fires.

Checkpoint

You're ready for Module 8 when your recurring task is one slash command long, you've run one real task through plan mode, and permission prompts in your main project only show up for genuinely unusual actions.

Module 8 Capstone: Build and Publish a Real Website

Time: about 90 minutes, entirely hands-on. You finish with a live URL.

Objectives

Ship a real website to the public internet using everything from Modules 1 through 7: briefing, plan mode, iteration, CLAUDE.md, git and GitHub, and verification. Without writing code yourself.

A web agency would quote four figures for what you're about to do in an afternoon. That's not a knock on agencies. That's the point of the last seven modules.

8.1 Choose your build

Pick whichever one is real for you. Real stakes make real learning. A site you actually care about will teach you twice as much as a throwaway.

- **A.** A personal or professional site (about, services or work, contact)
- **B.** A small-business site for someone you know (a real client!)
- **C.** A hobby or interest site (a guide, a collection, a local resource)

Whatever you pick, the bar is the same: at least 3 pages or sections, looks respectable on a phone, live on the internet at the end. No partial credit on that last one.

8.2 Stage 1: Brief and plan (use plan mode)

Make a new project folder, put it under git from minute one, and switch to plan mode. Then give the brief. Yours will look something like:

"I want a simple, fast, professional website for [X]. The audience is [who]. It needs: [pages/sections]. Tone and look: [for example, clean and warm, not corporate]. I'll deploy it free on Cloudflare Pages or GitHub Pages. Pick whichever is simpler for this project and tell me why.

Keep the tech as boring and low-maintenance as possible, because I'm a beginner maintaining this alone. Propose a plan including the folder structure and what each page contains."

Two points of technique, and the first one is worth the module by itself.

You named the *qualities* of the technology (boring, low-maintenance) rather than the technology. Claude picks the stack; you gave it the selection criteria. This is how a non-expert supervises expert decisions: in websites, in hiring, in everything. You don't need to know the answer. You need to know what a good answer has to satisfy.

Review the plan like a client, not an engineer. Are the pages right? Is anything missing? Does the maintenance story sound like something you'd actually keep up with?

Approve the plan and let it build. Then commit.

8.3 Stage 2: See it, shape it

"Run this site locally so I can see it in my browser."

You'll get a local address like `http://localhost:8080`. Open it. Now iterate the way you practiced in Module 3, one targeted correction at a time, checking the browser after each:

- "The headline is weak. Give me five alternatives with a more concrete benefit and less poetry."
- "Make the photos consistent in size and add breathing room between the sections."
- "Show me this at phone width. That menu is broken, fix it."

This stage is interior design, not construction. You point, it moves the couch. Commit at every state you'd be sad to lose. If an iteration makes things worse, "undo that last change." This is exactly why git came before the capstone. You get to experiment like it's free, because it is.

A note on the words: have Claude draft copy from your bullet points, but you own the final text. Read every sentence on the site out loud once. Drafted by AI, approved by a human. Your name is on this.

8.4 Stage 3: Pre-flight checks

Delegate the QA, then verify a sample of it yourself:

"Before we publish: check every link works, check spelling and grammar on all pages, confirm it looks right at phone, tablet, and desktop widths, make sure images are compressed for fast loading, and give me a plain-English report of anything questionable."

Read the report, then spot-check two of its claims yourself. Click a link. View the site at phone width. Trust it, but sample it. That habit, delegate the inspection, then inspect the inspector, is how you scale trust without gambling.

8.5 Stage 4: Publish

"Publish this. Walk me through anything that needs my accounts or my decisions, one step at a time."

What to expect: pushing to GitHub (you learned how in Module 6), connecting the host (Cloudflare Pages and GitHub Pages both deploy free, straight from a GitHub repository), and a couple of browser steps only you can do, like creating an account and clicking authorize. Claude narrates, you click.

Then the moment. A real URL. Open it on your phone, off your wifi. That's the actual internet. That's your site. You shipped it. Send the link to somebody.

Optional, about \$10 to \$15 a year: a custom domain. "I want this at [yourname].com. Walk me through buying the domain and connecting it." Fifteen minutes of DNS steps that Claude navigates for you.

8.6 Stage 5: Make it maintainable

Here's the difference between a project and a demo: what happens next month. Demos die the day after launch. Don't build a demo.

1. "Write a CLAUDE.md for this project: what it is, how it's structured, how deployment works, and the rules for changes (always check phone width, always commit before big changes, never break existing URLs)."
2. Learn the update loop for future-you: open the folder, run `claude`, describe the change, check it locally, commit, push, and it deploys automatically. Ask Claude to add that recipe to CLAUDE.md

too.

3. Test it. Close everything, come back later, ask for one small change (update a sentence, add an FAQ), and push it live. This loop is the real deliverable of the course: a durable ability, not a one-time artifact. The site is proof. The loop is the product.
-

Wrapping up the course

Look at the shape of what you just did. Brief with quality criteria. Plan before build. Iterate in small corrections. Commit at good states. Delegate QA but sample-check it. Publish. Document for future-you.

None of that is website-specific. That's the template for everything you'll do with an agent from here on: reports, automations, data work, the next tool you build. You didn't learn to make a website. You learned to run an employee that works for \$20 a month. The website was just the final exam.

Where to go next:

- Keep the Module 4 recognition checklist around for another two weeks. The reflex is the asset that compounds.
- When a service keeps making you export and copy-paste, revisit MCP (section 7.3). That's your trigger.
- Check `/help` and Anthropic's docs for what's changed since this course was written. The concepts stay stable. The features keep growing. You now have the skill to absorb new features on contact. That was the plan all along.

Final checkpoint

The course is complete when your site is live at a public URL, the repository is on GitHub with a real commit history (not one giant commit), CLAUDE.md documents the update loop, and you've done one post-launch change end to end.

Bonus Module: I Ran a Business Back Office on Claude Agents. Here's the Whole Thing.

Time: about 30 minutes reading, 45 more if you build the exercise. **Hands-on:** optional but recommended. This is a case study with the real files in it.

Why this module exists

Everything in modules 1 through 8 taught you skills. This module shows you what they stack into: a real business back office, sales research to outreach drafts, run by scheduled Claude agents while the owner did other things.

Full disclosure up front, because it makes the lesson better, not worse: the business this back office served never landed a paying client, and I eventually shut it down because the *delivery* side needed hours per client I didn't have. The automation worked. The business model didn't fit my life. You get to learn from both halves.

The business, in one paragraph

The service was a systems audit for small business owners: they pay a few hundred dollars, fill out a questionnaire, we do a discovery call, they get a written roadmap. Standard consulting-shaped offer. The bottleneck in any offer like this is the top of the funnel: finding prospects, researching whether they're a fit, and writing outreach that doesn't read like spam. Done by hand, that's hours per week of repetitive, judgment-light work. Which should be setting off your Module 7 alarm: any brief you'd type three times is a product waiting to be built.

The architecture: three agents and a spreadsheet

The whole system was an Airtable base (a spreadsheet with a nicer interface) plus three scheduled Claude agents, each one just a markdown file describing a job. That's it. No code, in the sense you feared before Module 1.

The pipeline:

1. **I drop names into an intake table.** People from my LinkedIn network, local business owners, referrals. Name, LinkedIn URL, status: Queued. Ten seconds each, done from my phone.

2. **Agent one: prospect research.** On a schedule, it checks the intake table for anything Queued, then for each prospect: reviews their public LinkedIn presence, searches for their website and social accounts, looks for evidence of the specific pains my service fixed (scheduling chaos, solo operation, posts about being overwhelmed), and scores fit from 1 to 5 against written criteria. Then it writes a full pipeline record: company, vertical, contact info if publicly findable, the evidence it found, and a one-sentence outreach angle referencing something real, a specific video title, a blog post, a booking page. It marks the intake row Complete and moves on.
3. **Agent two: fit-check briefs.** Before any sales call, it assembled everything known about the prospect into a one-page brief so I walked in prepared without prep time.
4. **Agent three: outreach drafts.** For researched prospects above a fit threshold, it drafted the personalized first-contact email using the research angle, saved as a draft for my review. Drafts, not sends. The human approves anything that leaves the building. That rule is not negotiable, and it's the same rule you learned about permissions in Module 7, scaled up to a business.

The actual agent file

Here's a condensed version of the real research agent, so you can see there's no magic. It's a brief. You could have written it after Module 3.

name: prospect-research

description: Automated prospect research

Check the Intake table in my Airtable base for records where Research Status equals Queued. If there are none, stop.

For each Queued record:

1. Set Research Status to In Progress.
2. Review their LinkedIn profile: title, company, team size, posts about systems, workflows, or overwhelm.
3. Search for their name and business. Check their website, YouTube, Instagram. Look for scheduling or onboarding pain, solo operation, revenue signals.
4. Score fit 1-5: solo or micro-team required for anything above 1; scheduling-heavy business required above 2; revenue signals add a point; public pain signal adds a point.

5. Write a one-sentence outreach angle referencing something REAL the research found. Generic angles score low.
6. Create a pipeline record with all fields filled.
7. Set Research Status to Complete. Continue to the next.

If research is inconclusive after a reasonable search, score 1, note the limitation, and move on.

Read that twice and notice what it is: a job description for a diligent assistant, with the judgment criteria written down. The scoring rubric matters most. "Score how good a fit they are" produces mush; "solo or micro-team is required for any score above 1" produces decisions you can audit.

What it was like to operate

Mornings, I'd open Airtable to a table of researched prospects, scored and sorted, each with an outreach angle grounded in something the agent actually found. The research that would have eaten my evenings happened without me. Real outputs from the real table: bookkeepers and photographers scored 3 to 5 with public pain signals quoted; enterprise employees who'd slipped into my intake list scored 1 and auto-archived with the reason noted. The filtering alone justified the setup, because the most expensive outreach is the well-written email to someone who was never going to buy.

Costs, honestly: a few sessions to set up the Airtable base and tune the agent briefs, then roughly zero marginal effort per prospect. The tuning was mostly tightening the scoring rules after early batches scored too generously. You'll recognize that loop from every module: run it, read the output, sharpen the brief.

Why I shut it down anyway, and what that teaches

The back office scaled beautifully. The service didn't: every client meant hours of calls and report writing from me personally, and my weeks didn't have those hours. Automation multiplies a business; it doesn't change what the business is. Multiply a time-for-money service by great automation and you get a very efficient way to sell your unavailable time.

The takeaways worth paying for:

1. **Automate the funnel, but check the engine.** Before building a back office, make sure the thing it feeds is something you can actually deliver at volume.

2. **Agents are briefs on a schedule.** Everything you learned about writing good briefs is the skill. The scheduling is a detail.
3. **Judgment criteria go in writing.** The 1-to-5 rubric is what separated useful automation from confident noise.
4. **Drafts, not sends.** Autonomy at the edges of your business, human approval at the boundary where it touches other people.
5. **A system you can shut down cleanly is a system that was built well.** When I killed the service, the agents were three files to archive and one table to flag. Build so that stopping is cheap, because some experiments should stop.

The exercise: build a one-agent back office

Pick a recurring research task in your own work. Prospects, competitors, grant deadlines, local permits, whatever you look up repeatedly. Then, in a session:

"I want a recurring agent that [your task]. Interview me about the judgment criteria until you could score results 1 to 5 without me, then write the agent file with those criteria in it, and set it up to run on a schedule. Its output goes to [a file, a spreadsheet], and it never contacts anyone or posts anything."

Note what you're doing in that prompt: making Claude extract your judgment into written rules before any automation exists. That interview is the exercise. The agent is just where the answers live.

Run it for two weeks. Tighten the rubric when the output disagrees with your gut, because your gut is usually detecting a rule you haven't written down yet.

That's the course. You came in wondering whether you could use a developer tool. You're leaving with a back office pattern that businesses pay consultants five figures to design. Go build the boring thing that gives you your evenings back.

Claude Code: One-Page Cheat Sheet

Everything from the course, one page. Keep this open. Stop guessing, start looking it up.

Launching

<code>claude</code>	Start in the current folder (the "job site")
<code>claude --continue</code>	Resume the most recent conversation here
<code>cd <folder> first</code>	Good habit, not a rule. The permission prompt is the real safety net

In-session commands

<code>/help</code>	The always-current manual
<code>/clear</code>	Fresh start. New task means new clear. CLAUDE.md survives.
<code>/compact</code>	Condense a long conversation you're not done with
<code>/model</code>	Switch brains. Escalate for hard or high-stakes tasks.
<code>/init</code>	Generate a CLAUDE.md for a code project
<code>Esc</code>	Interrupt mid-action. Always allowed.
<code>Shift+Tab</code>	Cycle modes: normal, auto-accept, plan mode
<code>/exit</code> or <code>Ctrl+C</code> twice	Exit (your files stay)
<code>#</code> prefix	Save a note to memory for future sessions

The brief (4 parts)

1. **Outcome:** what exists when the work is done
2. **Materials:** where the inputs are
3. **Constraints:** what must and must not happen
4. **Done:** how you'll both know it worked

For anything touching many files, add: "show me a plan and a small sample first."

Phrases that earn their keep

- "Show me a plan before touching anything."
- "What did you change and why?"
- "What could go wrong with this approach?"
- "Write a check that verifies this a different way, and run it."
- "Good, but..." (a targeted correction beats a restart)
- "Stop fixing. Diagnose first, then propose, then wait."
- "Add that to CLAUDE.md."
- "Commit this." / "Commit first, then let's try it."
- "Undo the changes since the last commit."
- "Turn this into a script / slash command I can reuse."

Git in English

Say	Get
"Put this project under version control"	git, .gitignore, first commit
"Commit this with a sensible message"	Save-point
"What changed since the last commit?"	Plain-English diff
"Undo changes to X, keep the rest"	Single-file restore
"Roll back to how it was Tuesday"	Time travel
"Create a private GitHub repo and push"	Off-machine backup

Safety defaults (non-negotiable)

- New tool or new territory: sandbox folder first
- Many files: sample before the full run
- Say "don't delete anything" and "never overwrite originals" explicitly
- Money, legal, or outbound email: independent verification, every time
- Loosen permissions in proportion to your undo ability. Git-tracked folder, relax. Unversioned originals, keep the prompts.

When stuck

1. Say specifically what's wrong, with an example
2. Separate diagnosis from action ("explain first, don't fix yet")
3. Undo, then `/clear`, then re-brief with what you learned
4. Escalate the model (`/model`) for genuinely hard problems

Is this a Claude Code task?

Repetitive across many items. Format-shifting. Rule-based tedium. Many sources into one document. Anything you'd do if it took 5 minutes but skip because it takes an hour. If you nodded at any of those, stop reading and go delegate it.

Glossary: Plain-English Definitions

No jargon you can't use. If a term shows up in the course and you're not sure what it means, it's here.

Agent: An AI that takes actions (files, commands, web) rather than just answering in text. Claude Code is an agent.

Auto-accept mode: A session mode where Claude acts without permission prompts. For low-stakes sandbox work.

Brief: Your instruction to Claude. Outcome, materials, constraints, and definition of done.

CLAUDE.md: A file Claude Code automatically reads at launch. Your standing brief: project facts, rules, and working-style preferences. The personal version lives at `~/ .claude/CLAUDE.md`.

CLI (command-line interface): A program you use by typing commands in a terminal.

Commit: A named save-point of your whole project in git. "Commit this" means save the current state permanently.

Context: Everything Claude knows in the current session: your messages plus whatever it has read and run. It's gone when the session ends, which is what CLAUDE.md and memory are for.

Diff: The precise list of what changed between two versions. Ask for it in English: "what changed since the last commit?"

Git: A save-point system for a folder. History, comparison, and restore for every file. You use it through Claude in plain English.

GitHub: A website that hosts a synced online copy of your git project. Backup, sharing, and the launch pad for free website deployment.

Hook: An automation that fires on an event ("after every edit, do X"). Rules enforced by the machine.

localhost: Your own computer acting as a website host. An address like `http://localhost:8080` is a preview only you can see.

MCP (Model Context Protocol): The plug-in standard that connects Claude Code to outside services like Gmail, Notion, and databases.

Model: The AI brain currently in use. Bigger models (Opus-class) think deeper. Smaller ones (Haiku-class) are faster and cheaper. `/model` switches between them.

Permission prompt: Claude asking before it takes an action with side effects. Your safety rail, and your window into what it's doing.

Plan mode: Claude researches and proposes but changes nothing until you approve. For big or fuzzy work.

Push: Send your local commits to GitHub. "Push my latest commits."

Repository (repo): A folder under git version control. On GitHub, the online copy of one.

Sandbox: A throwaway folder for practice and experiments, where nothing can be harmed.

Skill: A folder of instructions Claude loads for a specific kind of task (how we write proposals, how we review data).

Slash command: Anything starting with `/` typed in a session. There are built-ins like `/help` and `/clear`, plus your own canned briefs stored in `.claude/commands/`.

Subagent: A parallel copy of Claude that the main session delegates a piece of work to. Mostly automatic.

Terminal: The text window where you run commands and where Claude Code lives. PowerShell on Windows, Terminal on Mac.

Version control: The general practice git implements: a tracked history of changes with the ability to restore any of them.

Working directory: The folder you launched Claude Code in. Its job site. Easy access inside it, extra checks outside it.